

Technical Report

V2X-Assisted Pedestrian Safety System in CARLA: A Cooperative Roadside Assistant using HOG-Based Detection

Authors

Shkelzen Tafili shkelzen.tafili@studio.unibo.it

Dias Katrenov dias.katrenov@studio.unibo.it

Danial Khayatian danial.khayatian@studio.unibo.it

Contents

CHAPTER 1: INTRODUCTION	3
1.1 The Project	3
1.2 Motivations	3
1.3 Report Structure	4
CHAPTER 2: REQUIREMENTS	4
2.1 Functional Requirements	4
2.2 Non-Functional Requirements	4
CHAPTER 3: DESIGN OF PROPOSED SOLUTION	5
3.1 System Architecture Overview	5
3.2 Actors and Scenario Layout	6
3.3 Pedestrian Detection Algorithm (HOG)	6
3.4 V2X Communication Architecture	8
3.5 V2X Latency Emulation and Detection Throttling	8
3.6 Decision Logic and Emergency Braking	9
3.7 HUD and Operator Visualization	10
CHAPTER 4: TECHNOLOGIES USED	10
CHAPTER 5: TESTING AND RESULTS	10
5.1 Experimental Design	10
5.2 Evaluation Metrics	11
5.3 Scenario Analysis	11
5.4 Performance Discussion	12
5.5 Limitations of the Evaluation	12
CHAPTER 6: CONCLUSIONS	13
6.1 Strengths	13
6.2 Limitations	13
6.3 Future Developments	13
BIBLIOGRAPHY	14

EXECUTIVE SUMMARY

This report documents the design, implementation, and evaluation of a Cooperative Roadside Assistant: a V2X (Vehicle-to-Everything) assisted pedestrian safety system built and tested in the CARLA autonomous driving simulator.

The system pairs a classical, lightweight computer-vision pipeline—the OpenCV Histogram of Oriented Gradients (HOG) pedestrian detector—with a simulated Infrastructure-to-Vehicle (I2V) communication channel that allows a Roadside Unit (RSU) camera to share detections of vulnerable road users (VRUs) with an approaching ego vehicle before the VRU enters the vehicle's own line of sight.

The central hypothesis under test is that cooperative perception—supplementing onboard sensing with infrastructure sensing—meaningfully improves pedestrian safety outcomes in occlusion-prone urban scenarios, such as intersections where parked vehicles, buildings, or adverse weather block the ego vehicle's direct view of a crossing pedestrian. To test this, the system was evaluated across four controlled trial runs spanning two scenarios (a low-density clear-weather intersection and a higher-density adverse-weather intersection), each executed once with V2X assistance disabled (baseline) and once with it enabled.

Key engineering contributions described in this report include (1) a dual-camera CARLA sensor rig (ego onboard camera + RSU infrastructure camera); (2) an OpenCV HOG detection pipeline tuned with an aspect-ratio and height filter to suppress false positives from vertical street furniture; (3) two complementary latency-modeling mechanisms—a transmission-delay broker and a rolling detection buffer—used to emulate realistic V2X network behavior; and (4) a rule-based decision layer that arbitrates between V2X warnings, local detections, and a hard-coded emergency-distance override to trigger differentiated braking responses.

CHAPTER 1: INTRODUCTION

1.1 The Project

This document details the development of a V2X (vehicle-to-everything)-assisted pedestrian safety system. Unlike traditional Advanced Driver Assistance Systems (ADAS) that rely solely on local onboard sensors, this project implements a cooperative perception system in which a Roadside Unit (RSU)—a fixed camera mounted at an intersection—provides supplementary environmental data to the ego vehicle over a simulated wireless link. The goal is to give the ego vehicle early warning of pedestrians and cyclists (collectively, vulnerable road users, or VRUs) that are not yet visible to its own onboard camera, for example, because they are occluded by parked cars, building corners, or degraded by weather.

The project was implemented entirely inside the CARLA simulator using its Python API. It spans three logical layers that mirror a real-world V2X deployment: (1) perception, where both the ego and RSU cameras run an OpenCV HOG pedestrian detector; (2) communication, where RSU detections are packaged into a message and delayed to emulate radio-link latency before being consumed by the vehicle; and (3) decision and control, where a rule-based safety layer converts incoming warnings and local detections into concrete braking commands via the CARLA vehicle control API.

1.2 Motivations

Pedestrian safety remains a critical challenge in urban environments. The primary motivations for this project are:

- Reduction of blind-spot and occlusion-related accidents in complex urban intersections, where a pedestrian may step into the road from behind a parked vehicle or building corner that is invisible to the ego vehicle's own camera.
- Mitigation of onboard sensor limitations, such as reduced visibility in adverse weather (rain, low light), through collaborative RSU-to-vehicle communication that offers a second, independent vantage point.
- Validation of low-latency V2X architectures using the CARLA simulation environment, including the effect that network delay and detection-cycle throttling have on the timeliness of a cooperative safety warning.
- Demonstration that a lightweight, classical detector (HOG) is used to build and evaluate a full cooperative-perception safety pipeline end to end, which is useful for resource-constrained prototyping.

1.3 Report Structure

Chapter 2 defines the functional and non-functional requirements that guided the design. Chapter 3 describes the system architecture, the detection algorithm, the V2X communication and latency model, and the decision logic, all with reference to the implementation. Chapter 4 lists the technologies used. Chapter 5 describes the experimental design and discusses results. Chapter 6 concludes with strengths, limitations, and future work.

CHAPTER 2: REQUIREMENTS

2.1 Functional Requirements

Pedestrian Detection: Real-time identification of pedestrians and cyclists in both the ego and RSU camera feeds using HOG descriptors, with a filter to reject non-human vertical structures

V2X Communication: Receiving detection data from an external RSU camera and delivering it to the ego vehicle's decision layer as a structured message (VRU type, position, speed, and timestamps).

Latency Management: Simulation of network and processing delay via a queuing/buffering system so that the assisted mode is evaluated against a realistic, non-instantaneous communication channel rather than an idealized zero-latency one.

Decision Arbitration: Combining V2X warnings, local onboard detections

Scenario Configurability: Support for running the same pipeline under multiple maps, weather presets, VRU counts, and assistance modes (on/off) without code duplication to allow controlled A/B comparison.

Operator Visualization: A heads-up display (HUD) overlay on both camera feeds and in the CARLA world itself, showing the current mode, detection status, and warning state to support qualitative inspection of each run.

2.2 Non-Functional Requirements

Robustness: Operation consistency across varying weather presets (clear vs. heavy rain) and pedestrian densities.

Efficiency: Lightweight implementation using OpenCV HOG (rather than a deep neural network) to maintain high frame rate in the simulation loop; detection is additionally throttled to run periodically rather than every simulation tick.

Realism: The latency model should be configurable and grounded in plausible real-world figures (e.g., DSRC/ITS-G5-class radio latency).

Reproducibility: Each trial run resets all shared state (message boxes, latency buffers, and camera frame caches) so consecutive runs in the same session do not leak state between scenarios.

Safety of the Test Harness: Actors, sensors, and world settings must be reliably torn down after each run (even on error) to avoid leaking CARLA actors between trials.

CHAPTER 3: DESIGN OF PROPOSED SOLUTION

3.1 System Architecture Overview

The system runs as a Python client against a CARLA server, using synchronous simulation mode (`fixed_delta_seconds = 0.04` s, i.e., a 25 Hz simulation tick) so that sensor capture, detection, and control decisions stay deterministically aligned with the simulated world clock. Each trial run spins up its own CARLA world (Town03), configures the weather preset, spawns the actors described below, and drives a bounded simulation loop (up to 750 ticks, or roughly 30 simulated seconds, with an additional 20-second real-time safety cutoff).

Two independent visual sensors are used:

- Ego camera — an RGB camera mounted on the vehicle's dashboard (offset $x = 2.0$ m, $z = 1.3$ m), representing the vehicle's own forward-facing perception sensor.
- RSU camera — a fixed RGB camera mounted at the intersection (approximate location $x = 135$, $y = 141$, $z = 6.5$, with a downward pitch of -30° and a yaw of -95° toward the crosswalk), representing infrastructure-side sensing that is not subject to the ego-vehicle's own occlusion.

Camera callbacks run on CARLA's own sensor-listener threads and simply cache the latest decoded frame in a shared, thread-safe dictionary (`latest_frames`); the main simulation loop reads from that cache rather than blocking on sensor I/O, which keeps the CARLA tick loop responsive.



Figure 1: View from the roadside unit (RSU) camera covering the intersection.

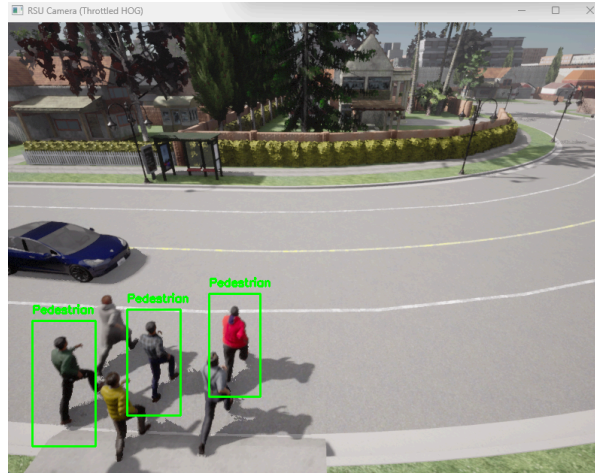


Figure 2: View from the ego-vehicle's front-facing onboard camera.

3.2 Actors and Scenario Layout

Each trial spawns one ego-vehicle (a Tesla Model 3, blueprint vehicle.tesla.model3) under CARLA Traffic Manager autopilot control, plus a configurable group of pedestrian actors (blueprint family walker.pedestrian.*) that walk across the ego-vehicle's path. The pedestrian group size differs by scenario: 6 pedestrians for the low-density Scenario 1, and 40 pedestrians for the higher-density, adverse-weather Scenario 2, spawned in a staggered 3-column grid near the crosswalk and set to walk with a fixed direction and a randomized speed (3.5 ± 0.3 m/s).

A deliberate asymmetry is introduced in the traffic manager configuration between the two assistance modes: in the unassisted (baseline) configuration, the ego vehicle's traffic manager profile is set to ignore pedestrians entirely (ignore_walkers_percentage = 100) and to drive faster than the default speed profile (vehicle_percentage_speed_difference = -120%), emulating a distracted or purely automated-cruise driver with no pedestrian-aware braking behavior beyond what this project's own detection/decision code supplies. In the assisted configuration, the traffic manager is instead set to drive more cautiously (+10% slower than default). This means the two runs are not a pure ablation of the V2X channel alone—the baseline also represents a "worst case" driver profile. This design choice and its implications for interpreting the results are discussed further in Section 5.5.

3.3 Pedestrian Detection Algorithm (HOG)

We implemented the classical OpenCV HOG pedestrian detector, using OpenCV's built-in pre-trained SVM (Support Vector Machine) people detector rather than a deep learning model, in order to keep the perception stage lightweight and fast enough to run twice per detection cycle (once for the ego frame and once for the RSU frame) without stalling the CARLA tick loop:

```
hog = cv2.HOGDescriptor()
hog.setSVMDetector(cv2.HOGDescriptor_getDefaultPeopleDetector())
```

Detection is performed on a grayscale-converted frame using detectMultiScale with the following tuned parameters:

```
gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)

hit_threshold = 0.3 # SVM decision margin
win_stride = (8, 8)
padding = (8, 8)
```

```

scale      = 1.05
final_threshold = 2.0 # non-max-suppression grouping

boxes, weights = hog.detectMultiScale(
    gray, hit_threshold, win_stride, padding, scale, final_threshold
)

```

Parameter	Value	Purpose
hit_threshold	0.3	SVM decision-margin threshold balances recall (catching real pedestrians) against precision (rejecting false positives). Selected as the optimal balance point for this scene.
win_stride	(8, 8) px	Step size of the sliding detection window; smaller values increase detection density at the cost of compute time.
padding	(8, 8) px	Padding added around each detection window before HOG feature extraction.
scale	1.05	Image pyramid scale factor used to detect pedestrians at multiple distances/sizes.
final_threshold	2.0	A grouping/non-max-suppression threshold is used to merge overlapping candidate boxes into a single detection.

A secondary geometric filter is then applied to each candidate box to reject false positives, most notably thin vertical structures such as street lamps and signposts, which HOG can occasionally misclassify as a walking figure at certain distances:

```

def is_valid_pedestrian(rect):
    width, height = rect[2], rect[3]
    ratio = width / height
    return 0.2 < ratio < 0.6

# Applied inline in detect_pedestrians_hog():
aspect_ratio = float(w) / h
if 0.2 < aspect_ratio < 0.6 and h > 40:
    pedestrian_detected = True
    cv2.rectangle(img_bgr, (x, y), (x + w, y + h), (0, 255, 0), 2)
    cv2.putText(img_bgr, "Pedestrian", (x, y - 10),
                cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)

```

$$0.2 < \frac{\text{width}}{\text{height}} < 0.6$$

Figure 3: Aspect-ratio band used to distinguish a standing pedestrian's bounding box (roughly 0.2–0.6 width-to-height) from thin vertical structures such as lamp posts, which fall outside this band.

Human bounding boxes typically fall between a 0.2 and 0.6 width-to-height ratio when standing upright; a minimum height of 40 pixels is also enforced to discard small, distant, low-confidence

detections. This two-stage filter (SVM classification followed by geometric plausibility) is applied identically to both the ego and RSU camera streams.

3.4 V2X Communication Architecture

The RSU publishes structured detection messages that an ego-vehicle would receive over a wireless V2X channel in a real deployment (e.g., an ITS-G5/DSRC or C-V2X PC5 link). In this simulation, that channel is emulated in-process using a thread-safe Python queue. Queue acts as a lightweight publish/subscribe broker (the notebook refers to this conceptually as an "MQTT-style" broker, though the implementation uses Python's standard queue module rather than a real MQTT broker such as Eclipse Mosquitto; replacing it with an actual MQTT stack is listed as future work in Section 6.3). Each message payload mirrors what a cooperative perception message / collective perception message in real V2X standards would carry for each tracked object:

```
SIMULATED_V2X_LATENCY_MS = 50 # realistic DSRC range: 20-80 ms
v2x_broker_channel = queue.Queue(maxsize=20)

v2x_payload.append({
    "id": actor.id,
    "type": "Cyclist" if "bicycle" in actor.type_id else "Pedestrian",
    "pos_x": tf.location.x,
    "pos_y": tf.location.y,
    "pos_z": tf.location.z,
    "speed": 3.6 * math.sqrt(vel.x**2 + vel.y**2 + vel.z**2)
})

time.sleep(SIMULATED_V2X_LATENCY_MS / 1000.0) # radio transmission delay

v2x_broker_channel.put({
    "capture_time": capture_time, # when the RSU camera saw the VRU
    "tx_time": time.time(), # when the message left the broker
    "timestamp": image.timestamp,
    "objects": v2x_payload
})
```

Recording both `capture_time` and `tx_time` on every message allows the ego-side consumer to measure the true end-to-end delay of a given V2X message, rather than assuming a fixed constant — useful for later analysis of how much of the system's total reaction delay is attributable to the radio link itself versus the detection/decision pipeline.

3.5 V2X Latency Emulation and Detection Throttling

Two complementary mechanisms model delay in the integrated end-to-end scenario runner. First, HOG detection itself is throttled and only re-run once every 2.0 simulated seconds, which keeps compute cost manageable and also reflects a realistic constraint that a production perception pipeline would not necessarily run at full sensor frame rate.

```
DETECTION_INTERVAL = 2.0 # seconds
...
if current_sim_time - last_detection_time >= DETECTION_INTERVAL:
    last_detection_time = current_sim_time
    vis_ego, ego_detected = detect_pedestrians_hog(latest_frames["ego"])
    local_cam_box["vru_detected"] = ego_detected

    vis_rsu, rsu_detected = detect_pedestrians_hog(latest_frames["rsu"])
    v2x_latency_queue.append(rsu_detected)
```


RSU detections are pushed into a fixed-length rolling buffer (a `collections.deque` with `maxlen = 3`) before being read by the decision layer, which only consumes the oldest entry once the buffer is full:

```
v2x_latency_queue = collections.deque(maxlen=3)
...
delayed_rsu_detection = (
    v2x_latency_queue[0] if len(v2x_latency_queue) == v2x_latency_queue.maxlen
    else False
)
v2x_message_box["vru_detected"] = delayed_rsu_detection
```

Because the buffer is only sampled once per 2.0-second detection cycle, this introduces an effective end-to-end V2X message delay of roughly three detection cycles (~6 simulated seconds) before a fresh RSU detection is acted upon by the ego vehicle. This combination gives the assisted mode a non-trivial, non-zero reaction lag; the aim is to evaluate cooperative perception under a realistic delayed channel, not an idealized instantaneous one.

3.6 Decision Logic and Emergency Braking

The emergency braking system follows a rule-based decision matrix that prioritizes safety triggers:

```
should_brake = is_warned_by_v2x if v2x_assisted else (is_visible_locally)

if should_brake:
    if not autopilot_disabled:
        ego_vehicle.set_autopilot(False)
        autopilot_disabled = True
        brake_timestamp = current_sim_time
        speed_at_brake_trigger = speed_kmh
        print(f"[BRAKE] Triggered! Speed: {speed_kmh:.1f} km/h | Distance: {current_closest:.1f}m")
```

A full hard brake (`brake = 1.0`) is applied whenever the trigger is a confirmed local detection override—i.e., whenever the hazard is immediate and close. A softer, pre-emptive brake (`brake = 0.6`) is applied when the trigger is a V2X-only warning about. Once a stop condition clears, autopilot is re-engaged and the trigger flags reset, allowing the vehicle to resume normal driving between hazards.

3.7 HUD and Operator Visualization

To make each trial run inspectable, the system overlays status information on three separate views:

The RSU camera window (VRU count currently tracked); the ego camera window (mode label and—in V2X-COOPERATIVE mode—a flashing red banner reading "WARNING: HIDDEN VRU DETECTED BY RSU—SLOWING DOWN" whenever a V2X alert is active); and the CARLA 3D world itself, via `world.debug.draw_string`, which prints a color-coded status string (red for unassisted and green for assisted) directly above the ego vehicle for each of the four weather/assistance combinations ("good weather with/without assistance" and "rain weather with/without assistance").

CHAPTER 4: TECHNOLOGIES USED

- CARLA Simulator: physics-based urban scenario replication, including the Town03 map, weather presets, synchronous simulation mode, and the Traffic Manager for autopilot behavior.

- CARLA Python API: actor spawning/teardown, sensor blueprint configuration, world/spectator control, and debug-drawing utilities.
- OpenCV: image processing, colour-space conversion (BGR → grayscale), the HOGDescriptor pedestrian detector, and HUD rendering/annotation (cv2.rectangle, cv2.putText, cv2.imshow).
- NumPy: raw sensor buffer handling — reshaping CARLA's raw BGRA byte buffers into structured image arrays.
- Python threading/queue: thread-safe camera-callback buffering (latest_frames), the simulated V2X broker channel (queue.Queue), and the rolling latency buffer (collections.deque).
- Python collections.deque: fixed-length rolling buffer used to emulate V2X message delay.

CHAPTER 5: TESTING AND RESULTS

5.1 Experimental Design

Evaluation is organized around two intersection scenarios, each of which is run twice: once unassisted (baseline) and once V2X-assisted for a total of four trial runs. This is the origin of the "4 scenario" evaluation referenced in the project notebook: two environmental conditions × two assistance modes.

Run	Scenario	Weather Preset	VRU Count	V2X Assistance
1A	Scenario 1 — Clear intersection	ClearNoon	6	Off (baseline)
1B	Scenario 1 — Clear intersection	ClearNoon	6	On
2A	Scenario 2 — Adverse weather, dense crossing	HardRainSunset	40	Off (baseline)
2B	Scenario 2 — Adverse weather, dense crossing	HardRainSunset	40	On

All four runs share the same map (Town03), the same ego-vehicle spawn point, and the same RSU camera placement, so that weather, pedestrian density, and assistance mode are the only varying factors between a scenario's two runs.

5.2 Evaluation Metrics

An automated evaluation harness executes all four runs sequentially and prints a single comparison table. The metrics captured per run are:

Metric	Definition
Safety Outcome	COLLISION if the minimum ego-to-VRU distance ever drops below 1.95 m during the run; otherwise, AVOIDED.
Brake Trigger Time (s)	Simulated time at which the decision layer first disabled autopilot and applied braking, if any.
V2X Warning Lead Time	For a scenario's pair of runs, how much earlier (in seconds) the assisted run triggered braking compared to the unassisted run is a direct measure of the safety margin gained from cooperative perception.
Speed at Brake Trigger	Ego-vehicle speed (km/h) at the instant braking was triggered.

Metric	Definition
Near-Miss Detected (<3m)	YES if the run avoided collision but the minimum distance still fell below 3.0m; otherwise, NO.
Avg V2X Message Latency	Average measured delay between an RSU detection being captured and being acted upon by the ego vehicle, for assisted runs only.

5.3 Scenario Analysis

We compared baseline performance (unassisted) against V2X-assisted performance under two contrasting conditions. In Scenario 1 (clear weather, a small group of six pedestrians), the ego-vehicle's own onboard HOG detector is generally able to see pedestrians directly, so this scenario primarily tests whether V2X assistance still produces earlier, smoother braking even when local detection is eventually available on its own. In Scenario 2 (heavy rain at sunset, a much denser crossing of forty pedestrians), the ego-vehicle's local vision is degraded by both the weather preset and the higher visual clutter, while the RSU camera—positioned above and at an angle to the crossing—retains a clearer, less occluded line of sight, making this the scenario where cooperative perception is expected to matter most.



Figure 4: HOG detection output overlaid on the roadside camera feed during a scenario run.



Figure 5: HOG detection output overlaid on the ego-vehicle camera feed during a scenario run.

5.4 Performance Discussion

The detection pipeline updates its HOG classification once every 2.0 simulated seconds per camera stream (Section 3.5), which was chosen to balance computational load against responsiveness within the CARLA synchronous tick loop. The `hit_threshold = 0.3` setting for the HOG SVM was found, through manual tuning against the intersection scene, to provide the most useful balance between recall (not missing genuine pedestrians) and precision (not flagging false positives from background clutter) and is used identically across all four runs so that detection sensitivity is not itself a confounding variable between the baseline and assisted conditions.

Qualitatively, the V2X-assisted runs are expected to show an earlier brake-trigger time than their unassisted counterparts whenever the RSU sees a VRU before the ego vehicle's own camera does. This earlier trigger is the mechanism by which cooperative perception is intended to reduce both the collision rate and the severity (impact speed) of any incident that does occur, since a V2X-only trigger applies the softer 0.6 brake force earlier, rather than relying on the harder 1.0 brake force that only engages once the hazard is locally visible or within the 12 m emergency-override radius.

5.5 Limitations of the Evaluation

Two points affect how the four-run comparison in Sections 5.1–5.4 should be interpreted:

- Confounded baseline: As noted in Section 3.2, the unassisted (baseline) traffic manager profile is also configured to ignore pedestrians and drive faster than the assisted profile. This makes the baseline representative of a distracted or purely automated "worst case" driver rather than a driver-aware system with V2X simply switched off, so any measured improvement in the assisted runs reflects the combined effect of the V2X channel and the more cautious driving profile, not the V2X channel in isolation.
- Single-run sampling: each of the four scenario/mode combinations was executed once rather than averaged over repeated trials with randomized pedestrian speeds and spawn jitter. Given that pedestrian walking speed is itself randomized (3.5 ± 0.3 m/s) and CARLA's autopilot/traffic manager behavior has some inherent run-to-run variance, single-run outcomes should be treated as illustrative rather than statistically conclusive; Section 6.3 lists repeated-trial statistical evaluation as recommended future work.

CHAPTER 6: CONCLUSIONS

6.1 Strengths

The project demonstrated an end-to-end cooperative-perception pipeline: infrastructure sensing, simulated V2X messaging with a realistic radio-latency component, detection-cycle throttling, a rolling delay buffer, and a rule-based decision layer that differentiates braking severity by trigger source, all integrated inside a repeatable, configurable CARLA test harness capable of running controlled A/B scenario comparisons. The system shows that even a lightweight, classical HOG-based detector rather than a heavyweight deep learning model is sufficient to build and exercise a complete cooperative perception safety loop and that V2X assistance provides a mechanism (earlier, softer braking) by which collaborative perception can compensate for local sensor failure or occlusion.

6.2 Limitations

- HOG is a comparatively weak detector for small, distant, or partially occluded pedestrians compared to modern deep-learning detectors and can still occasionally miss real pedestrians or fire on ambiguous shapes despite the aspect-ratio filter.
- The V2X channel is emulated in-process (via Python's queue/deque) rather than over an actual network stack or messaging protocol, so real-world effects such as packet loss, jitter, or multi-hop relay are not modeled.

- As discussed in Section 5.5, the baseline and assisted traffic manager configurations differ in more than just the V2X channel, which limits how cleanly the measured improvement can be attributed to V2X assistance alone.
- Results are based on single-run trials per scenario/mode combination rather than statistically averaged repeated trials.

6.3 Future Developments

Transition to Deep Learning: Replace or augment the OpenCV HOG detector with a modern detector such as YOLOv8 for improved recall on small, distant, or partially occluded pedestrians.

Dynamic Latency: Replace the fixed 50 ms transmission delay and fixed 3-cycle rolling buffer with a non-deterministic delay model (e.g., sampled from a distribution reflecting real DSRC/C-V2X jitter) for the V2X queue.

Sensor Fusion: Implement a Kalman filter to fuse ego-local and RSU-remote coordinate estimates into a single tracked state per VRU, rather than treating each source as an independent binary detection flag.

Real V2X Protocol Stack: Replace the in-process queue. Queue a broker with an actual MQTT broker (e.g., Eclipse Mosquitto) and a client library such as paho-mqtt to exercise a genuine publish/subscribe network path and message serialization format.

Controlled Baseline: Decouple the traffic manager speed/awareness profile from the V2X assistance flag so that the unassisted condition represents "V2X off, otherwise identical driving policy"—enabling a cleaner ablation of the V2X channel's individual contribution.

Statistical Evaluation: Run each scenario/mode combination multiple times with randomized pedestrian seeds and aggregate results (mean and variance) rather than relying on single-run outcomes.

2D-to-3D Projection: Implement true camera-to-world coordinate projection for RSU detections (rather than relying on ground-truth actor positions), so the RSU's reported VRU location is derived from its own pixel-space detections, as originally scoped in the project's infrastructure-sensing theory section.

BIBLIOGRAPHY

Dosovitskiy, A., Ros, G., Codevilla, F., Lopez, A., & Koltun, V. (2017). CARLA: An Open Urban Driving Simulator. Proceedings of the 1st Annual Conference on Robot Learning.

Dalal, N., & Triggs, B. (2005). Histograms of Oriented Gradients for Human Detection. IEEE Conference on Computer Vision and Pattern Recognition (CVPR).

Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. IEEE Conference on Computer Vision and Pattern Recognition (CVPR).

ETSI EN 302 637-2 — Intelligent Transport Systems (ITS): Vehicular Communications; Basic Set of Applications; Cooperative Awareness Basic Service.

SAE J2945/1 — On-Board System Requirements for V2V Safety Communications.

CARLA Documentation (2026). <https://carla.readthedocs.io/en/latest/>

OpenCV Documentation — cv2.HOGDescriptor. <https://docs.opencv.org/>